typesense

Tips for Searching Common Types of Data

Copy Markdown

In this article we'll talk about how to index and search the following types of data:

- Model Numbers / Part Numbers / SKUs
- Phone Numbers
- Email Addresses
- Dates / Times
- Nested Objects
  - From Typesense v0.24.0
  - Typesense v0.23.1 and earlier
- Geographic Coordinates
- Long Pieces of Text
- HTML Content
- Searching for null or empty values
- URLs or File Paths
- Other Types of Data

Model Numbers / Part Numbers / SKUs

Let's say you have a document that contains a product identifier (model number, part number or SKU) with a mix of alphanumeric characters and special characters:

```json
{
  "title": "Control Arm Bushing Kit",
  "part_number": "K83913.39F29.59444AT"
  //...
}
```

Now let's say you want this product to show up in the search results for any of the following search terms:

- `K83913`

- 59444AT
- 59444
- 9444AT
- K83913.39F29
- 39F29.59444

Default Behavior

By default, Typesense removes special characters from fields when indexing and searching for them. So `K83913.39F29.59444AT` will get indexed as `K8391339F2959444AT`.

By default, Typesense does a Prefix Search, meaning that it only searches for records where the search term is at the *beginning* of strings. So searching for `39F29` or `F29` which occurs in the middle of `K83913.39F29.59444AT` will not pull up that record. But searching for `K83913` or `K83913.39` or `K83913.39F29.59444` or `K83913.39` will pull up that record.

Fine-Tuning

The first change we'd need to do is to tell Typesense to split the product identifier by `.` (period). This way `K83913.39F29.59444AT` will get indexed as three separate tokens (words) `K83913`, `39F29` and `59444AT`. Now when you search for `39F29` or `5944` that will return the product `K83913.39F29.59444AT`.

You can do this by setting `token_separators` in the schema when creating the collection:

```json
{
  "name": "products",
  "fields": [
    {"name":  "title", "type":  "string"},
    {"name":  "part_number", "type":  "string"}
  ],
  "token_separators": ["."]
}
```

> **Field-Level Control**
>
> From Typesense `v28.0` and above, you can configure `token_separators` and `symbols_to_index` at the field level, which takes precedence over collection-level

```json
{
  "name": "products",
  "fields": [
    {
      "name": "part_number",
      "type": "string",
      "token_separators": ["-", "."],
      "symbols_to_index": ["_"]
    }
  ]
}
```

We still have the case of searching for `83913` or `9444AT` which occur in the middle of strings.

To solve for this, we have two options:

1. Use the new `infix` search feature available as of `v0.23.0`:

   https://github.com/typesense/typesense/issues/393#issuecomment-1065367947

   Note: For long strings, this might be a computationally intensive operation. If you notice that there is increased CPU usage for your particular use-case, you want to then use the option below.

2. Pre-split the product identifier based on how you expect your users to search for them:

```json
{
  "title": "Control Arm Bushing Kit",
  "part_number": [
    "K83913.39F29.59444AT",
    "83913.39F29.59444AT",
    "3913.39F29.59444AT",
    "913.39F29.59444AT",
    "13.39F29.59444AT",
    "3.39F29.59444AT",
    "9F29.59444AT",
    "F29.59444AT",
    "29.59444AT",
    "9.59444AT",
```

```
        "4AT",
        "AT"
      ]
      //...
    }
```

When you use this in conjunction with `token_separators`, you'll be able to search all the patterns we discussed above.

Phone Numbers

Let's say we have phone numbers in this format: `+1 (234) 567-8901` and we want users to be able to use any of the following patterns to be able to pull this record up:

- `8901`
- `567-8901`
- `567 8901`
- `5678901`
- `234-567-8901`
- `(234) 567-8901`
- `(234)567-8901`
- `1-234-567-8901`
- `+12345678901`
- `12345678901`
- `2345678901`
- `+1(234)567-8901`

Default Behavior

By default, Typesense will remove all special characters, and split tokens (words) by spaces and so `+1 (234) 567-8901` will be indexed as `1`, `234`, `5678901`.

So searching for `234` or `5678901` or `234 567-8901` will return results, but the other patterns will not return the expected result.

Fine Tuning

```json
{
  "name": "users",
  "fields": [
    {"name":  "first_name", "type":  "string"},
    {"name":  "phone_number", "type":  "string"}
  ],
  "token_separators": ["(", ")", "-"]
}
```

This will cause `+1 (234) 567-8901` to be indexed as `1`, `234`, `567` and `8901` and now the following searches will return this document:

- `8901`
- `567-8901`
- `567 8901`
- `234-567-8901`
- `(234) 567-8901`
- `(234)567-8901`
- `1-234-567-8901`
- `+1(234)567-8901`

The remaining cases to handle are:

- `5678901`
- `+12345678901`
- `12345678901`
- `2345678901`

To solve for these, you want to add these additional formats as a `string[]` array field in your document:

```json
{
  "name": "users",
  "fields": [
    {"name":  "first_name", "type":  "string"},
    {"name":  "phone_number", "type":  "string[]"}
  ],
  "token_separators": ["(", ")", "-"]
```

```json
{
  "name": "Tom",
  "phone_number": [
    "+1 (234) 567-8901",
    "12345678901", // Remove all spaces
    "2345678901", // Remove all spaces and country code
    "5678901" // Remove all space, country code and area code
  ]
}
```

Now, searching for any of the patterns above will pull up this record.

Email Addresses

Let's say we have an email address like `contact+docs-example@typesense.org` and we want users to be able to use any of the following patterns to be able to pull this document up:

- `contact+docs-example`
- `contact+docs-example@`
- `contact+docs-example@typesense`
- `contact+docs`
- `contact docs`
- `docs example`
- `contact typesense`
- `contact`
- `docs`
- `example`
- `typesense`
- `typesense.org`

Default Behavior

By default, Typesense will remove all special characters during indexing and only does a prefix search (search terms should be at the beginning of words), so `contact+docs-example@typesense.org` will be indexed as `contactdocsexampletypesense.org`.

- ✅ contact+docs-example
- ✅ contact+docs-example@
- ✅ contact+docs-example@typesense
- ✅ contact+docs
- ❌ contact docs
- ❌ docs example
- ❌ contact typesense
- ✅ contact
- ❌ docs
- ❌ example
- ❌ typesense
- ❌ typesense.org

Fine Tuning

To solve for the remaining cases above, we can use the `token_separators` setting in the schema when creating the collection:

```json
{
  "name": "users",
  "fields": [
    {"name":  "first_name", "type":  "string"},
    {"name":  "email", "type":  "string"}
  ],
  "token_separators": ["+", "-", "@", "."]
}
```

This will cause `contact+docs-example@typesense.org` to be indexed as `contact`, `docs`, `example`, `typesense` and `org`.

Now all the search terms will pull this document up:

- ✅ contact+docs-example
- ✅ contact+docs-example@
- ✅ contact+docs-example@typesense
- ✅ contact+docs
- ✅ contact docs

- ✅ `contact`
- ✅ `docs`
- ✅ `example`
- ✅ `typesense`
- ✅ `typesense.org`

If you also want `ample` to return this record, you can use the `infix` search feature available as of `v0.23.0` : https://github.com/typesense/typesense/issues/393#issuecomment-1065367947

Dates / Times

Typesense does not have a native date/time data type.

So you would have to convert dates and times to Unix Timestamps as described here.

Nested Objects

From Typesense v0.24.0

Typesense v0.24.0 supports nested objects and arrays of objects natively.

To enable nested fields, you'll need to use the `enable_nested_fields` property when creating the collection, along with the `object` or `object[]` data type:

```json
{
  "name": "docs",
  "enable_nested_fields": true,
  "fields": [
    {"name": "person", "type": "object"},
    {"name": "details", "type": "object[]"}
  ]
}
```

Read more here.

Typesense v0.23.1 and earlier

for fields in the collection, which are the ones that will be indexed.

Important Side Note: You can still send nested objects into Typesense, in fields not mentioned in the schema. These will not be indexed or type-checked. They will just be stored on disk and returned if the document is a hit for a search query.

Typesense specifically does not support *indexing*, *searching* or *filtering* nested objects, or arrays of objects. We plan to add support for this shortly as part of (#227). In the meantime, you would have to flatten objects and arrays of objects into top-level keys before sending the data into Typesense.

For example, a document like this containing nested objects:

```json
{
  "nested_field": {
    "field1": "value1",
    "field2": ["value2", "value3", "value4"],
    "field3": {
      "fieldA": "valueA",
      "fieldB": ["valueB", "valueC", "valueD"]
    }
  }
}
```

would need to be flattened as:

```json
{
  "nested_field.field1": "value1",
  "nested_field.field2":  ["value2", "value3", "value4"],
  "nested_field.field3.fieldA": "valueA",
  "nested_field.field3.fieldB": ["valueB", "valueC", "valueD"]
}
```

before indexing it into Typesense.

To simplify traversing the data in the results, you might want to send both the flattened and unflattened version of the nested fields into Typesense, and only set the flattened keys as indexed in the collection's schema and use them for search/filtering/faceting. At display time when parsing the results, you can then use the nested version.

Geographic Coordinates

Typesense supports GeoSearch queries using latitude/longitude data in your documents. You can filter documents in a given radius around a lat/lng, or sort results by closeness to a given lat/lng or return results within a bounding box.

Here's more information about GeoSearch queries: GeoSearch API Reference.

Long Pieces of Text

If you have long pieces of text, like say a long journal article, website pages, transcripts, etc, we'd recommend that you break down the long piece of text into smaller "paragraphs" and store each paragraph in a separate document in Typesense.

This helps increase the granularity of search results and improve relevancy, because otherwise with sufficiently long text, there could be enough overlap in keywords between the documents, that searching for common keywords ends up matching most articles.

HTML Content

If you're searching HTML content, you want to create a field in your document which contains just the plain text version of the content without HTML tags and use that field in the `query_by` search parameter.

You can still store the raw HTML field in the document as an un-indexed field (by just leaving it from the schema), so the raw HTML will be returned in the document when it is a hit.

Here's more context around this.

Searching for `null` or empty values

Typesense does not have a way to filter documents that have `null` or empty values for an attribute, natively.

But you can still achieve this, using the following approach.

```json
{
  "tags": null
}
```

If you want to fetch all documents that have a `tags` set to `null`, you want to first create an additional field at indexing time in each document called `is_tags_null: true | false`:

```json
[
  {
    "tags": null,
    "is_tags_null": true
  },
  {
    "tags": ["tag1", "tag3"],
    "is_tags_null": false
  }
]
```

Once you've set this field in all your documents at indexing time, you can then query for these documents using:

```json
{
  "filter_by": "is_tags_null:true"
}
```

URLs or File Paths

Let's say you have documents with a set of URLs or file paths that you want to search on like this:

```json
{"url": "https://url1.com/path1"}
{"url": "https://url2.com/path2"}
{"url": "https://url3.com/path3"}
```

And you want Typesense to return results when users search for `url1` or `path1`, etc.

Default Behavior

the word) so `url1` or `path1` will not return any results, since they occur in the middle of the indexed string.

Fine-tuning

To solve for this and still fetch results for `url1` or `path1`, you want to add `:`, `.` and `/` to the `token_separators` setting in the collection schema:

```json
{
  "name": "pages",
  "fields": [
    {"name":  "title", "type":  "string"},
    {"name":  "url", "type":  "string"}
  ],
  "token_separators": [":", "/", "."]
}
```

This will now cause the URL to be indexed as separate words: `https`, `url1`, `com`, `path1`.

Now when you search for `url1` or `path`, it will match those individual words and return the document.

Other Types of Data

If you have other specific types of data that you'd like help with indexing in Typesense, please open a GitHub issue or ask in our Slack Community.

Edit page

Last Updated: 1/30/2025, 8:00:18 AM

← Tuning Ranking and Relevance Tips for Locale-Specific Search →